

# Zustandsgebundene Autorisierung

## Technische Unterlagen für die Patentberatung

Vertraulich · Fassung 1.0 · 20. September 2026

Eine Freigabe wird an den geprüften Verlauf, einen definierten Zustand und die ausgewählte Implementierung gebunden. Vor einer lokalen Testpublikation werden diese Werte und die Providererklärung erneut geprüft. Ein ausführbarer Prototyp und aufbewahrte Rohdaten belegen das beschriebene Verhalten im angegebenen Modell.

Dieses Paket erläutert das technische Problem, den Lösungsansatz, die Architektur, mögliche Anwendungen außerhalb des Repositorys und die konkreten Implementierungsreferenzen. Es dient der anwaltlichen Beurteilung des technischen Gegenstands; die abschließende patentrechtliche Prüfung bleibt offen.

Belegter Stand: 18 bestandene Dateisystemversuche, ein erfolgreicher persönlicher Demonstrationslauf und 640 bestandene Repository-Tests. Anwendungen in KI- und Agentensystemen sind als noch nicht implementierte Übertragungsentwürfe beschrieben.

Referenzcommit: `56ed0c743121fddc9d117b69eb470493dff5046d`  
Repository: `joku-dev/devsecops-governance-framework` · privater PR 176

## Leseführung

|                                                   |    |
|---------------------------------------------------|----|
| 1 Technische Problemstellung                      | 2  |
| 2 Lösungsansatz                                   | 4  |
| 3 Design und Architektur                          | 7  |
| 4 Anwendungen außerhalb der Repository Governance | 13 |
| 5 Implementierungsreferenzen und Nachweise        | 16 |
| 6 Herkunft und Fragen für die anwaltliche Prüfung | 21 |

Zusätzliche Dateien im Weitergabepaket: editierbare Kapitel, SVG-Abbildungen, Quellcodeauszug, originale Evidence-Pakete und eine lesende Integritätsprüfung. Die GitHub-Verweise sind auf feste Quellstände gebunden und erfordern privaten Zugriff.

# 1 Technische Problemstellung

## 1.1 Gegenstand und technisches Gebiet

Gegenstand ist die computerimplementierte Bindung einer Freigabe an einen genau bestimmten Ausführungskontext. Das Verfahren richtet sich an Systeme, die eine Aktion vorbereiten, eine gesonderte Zustimmung einholen und die Aktion erst später ausführen. Zwischen diesen Schritten können sich Daten, Berechtigungen oder ausführende Software ändern. Die technische Aufgabe besteht darin, eine veraltete Freigabe vor einem geschützten Schreibvorgang zuverlässig zu erkennen und die tatsächlich ausgeführte Zustandsänderung nachprüfbar aufzuzeichnen.

Der vorliegende Funktionsprototyp gehört zum privaten Repository

`joku-dev/devsecops-governance-framework`. Dieses verarbeitet technische Prüfergebnisse und daraus abgeleitete Governance-Zustände. Die hier untersuchte Lösung ist als eigener Forschungsbereich implementiert. Der nachgewiesene Effekt ist die Veröffentlichung eines lokalen Testartefakts in einem Transaktionsverzeichnis. Kapitel 3 beschreibt den ausführbaren Vertrag; Kapitel 5 ordnet jede wesentliche Funktion dem Quellcode und den Nachweisen zu.

## 1.2 Das konkrete technische Problem

Eine Zustimmung ist nur dann für eine spätere Aktion aussagekräftig, wenn klar ist, welche Eingaben, welche Vorgeschichte, welche aktuelle Zustandsableitung und welche Implementierung die zustimmende Stelle zugrunde gelegt hat. Eine Bestätigung allein des Aktionsnamens, eines Tickets oder eines früheren Berichts kann diesen Zusammenhang verlieren. Die gespeicherte Zustimmung kann unverändert bleiben, obwohl das System inzwischen eine andere technische Situation verarbeitet.

Ein Beispiel: Eine Person prüft einen Vorgang auf Grundlage des Prüflaufs E1 und der Auswertungssoftware I1. Während die Zustimmung eingeholt wird, geht Prüflauf E2 ein oder die Auswertungssoftware wird auf I2 geändert. Der spätere Ausführungsprozess findet weiterhin eine Zustimmung zum Vorgang. Ohne erneute Bindungsprüfung könnte er sie für einen Kontext verwenden, den die Person nicht geprüft hat. Auch zwei parallel gestartete Prozesse können dieselbe Freigabe verwenden und einen Effekt mehrfach erzeugen.

Die technische Problemformulierung lautet: Wie kann ein ausführendes System unmittelbar vor einer Zustandsänderung feststellen, dass der vollständige freigegebene Kontext weiterhin dem tatsächlichen Kontext entspricht, und die Prüfung mit dem beobachtbaren Schreibeffect so verbinden, dass konkurrierende lokale Ausführungen keine zweite Publikation unter demselben alten Antrag erzeugen?

Diese Fehlerklasse ist mit Time-of-check/Time-of-use-Problemen verwandt: Ein geprüfter Zustand kann sich zwischen Prüfung und Nutzung ändern. Diese allgemeine Fehlerklasse ist bekannt und wird hier nicht als neue Erkenntnis beansprucht. [E1]

## 1.3 Drei unterschiedliche Arten von Kontext

**Historie:** Welche konkrete geordnete Folge akzeptierter Transaktionen liegt vor? Eine zusätzliche Auditnotiz verändert diese Folge auch dann, wenn sie die sachliche Bewertung nicht verändert.

**Semantischer Zustand:** Welche für die Aktion festgelegten Werte entstehen aus einer deterministischen Verarbeitung der Historie? Im Prototyp sind dies unter anderem die letzte Evidenz, die aktive Publikation, die Rollenbindung und das Profil. Der Begriff bezeichnet eine ausdrücklich definierte Datenprojektion; er umfasst nicht automatisch jede denkbare Bedeutung der Quelldaten.

**Implementierung:** Welche ausgewählten Dateien, Abhängigkeitsdeklarationen und Konfigurationsbytes bestimmen die geprüfte Ausführungslogik? Gleiche Eingangsdaten allein reichen nicht aus, wenn eine geänderte Auswertung oder Konfiguration einen anderen Effekt erzeugen kann.

Die technische Schwierigkeit besteht darin, diese drei Identitäten auseinanderzuhalten und zugleich gemeinsam an die konkrete Aktion zu binden. Ein Hash des gesamten historischen Datenbestands erlaubt beispielsweise keine getrennte Aussage darüber, ob zwei Historien denselben definierten Zustand erzeugen.

## 1.4 Fehlerszenarien und gewünschtes Verhalten

| Szenario                                                  | Technisch gewünschtes Verhalten                                    |
|-----------------------------------------------------------|--------------------------------------------------------------------|
| Nach der Zustimmung wird neue Evidenz akzeptiert          | Alten Antrag zurückweisen                                          |
| Nur die aufbewahrte Historie ändert sich                  | Nach dem strikten Vertrag ebenfalls zurückweisen                   |
| Ausführungsrelevante Datei oder Konfiguration ändert sich | Geänderte Implementierungsbindung erkennen                         |
| Aktionsinhalt wird nachträglich ausgetauscht              | Fehlende Übereinstimmung mit der Erklärung erkennen                |
| Rolle, Profil oder Providererklärung ändern sich          | Vor einer neuen Publikation zurückweisen                           |
| Zwei Prozesse verwenden denselben Antrag                  | Höchstens eine Publikation im gemeinsamen lokalen Ledger           |
| Eine Korrektur wird später erforderlich                   | Historische Daten erhalten und aktuelle Wirksamkeit gezielt ändern |

## 1.5 Vorhandene Schutzmechanismen und verbleibende Untersuchungsfrage

Das Ausgangsrepository besitzt bereits Request-Digests, Prüfungen auf erwartete Historien-Heads und Sequenzen, Providerprüfungen, Implementierungsmanifeste sowie einen atomaren lokalen Writer. Diese Eigenschaften sind ein relevanter Ausgangspunkt. Der Prototyp ersetzt sie nicht durch eine Behauptung, frühere Freigaben seien ungebunden gewesen.

Untersucht wird ihre ausdrückliche Zusammenführung mit getrennt typisierten History-, State- und Implementation-Commitments sowie ihrer erneuten Auswertung an derselben lokalen Publikationsgrenze. Ein zusätzliches History-Commitment kann gegenüber einer korrekt geprüften Head-Bindung redundant sein. Diese Redundanz wird durch einen Weglassversuch sichtbar. Die zu prüfende mögliche Besonderheit liegt daher in einer konkreten technischen Merkmalskombination und deren Ausgestaltung, nicht in der isolierten Verwendung eines Hashwerts.

## 1.6 Abgrenzung der Aufgabenstellung

Der Nachweis betrifft weder die fachliche Richtigkeit einer Freigabe noch die Wahrheit der Eingangsevidenz. Er attestiert keine geladene Maschineninstruktion und beweist keine körperliche Anwesenheit einer Person. Ebenso wenig demonstriert er bereits eine verteilte Transaktion mit einem Deployment-System oder einen Betrieb für 1.500 Repositories. Diese Grenzen bestimmen, welche technischen Aussagen sich aus den Versuchen ableiten lassen.

## 2 Lösungsansatz

### 2.1 Kerngedanke

Die Lösung bildet drei getrennte kryptografische Commitments über die validierte Historie, eine deterministische Zustandsprojektion und ein explizites Implementierungsmanifest. Ein unveränderlicher Antrag enthält diese Commitments zusammen mit der konkreten Aktion, ihrem Inhalt, dem Geltungsbereich und den maßgeblichen Rollen- und Profilwerten. Die Zustimmung referenziert den Digest dieses vollständigen Antrags.

Vor der Publikation lädt das System den maßgeblichen Kontext erneut, rekonstruiert den Zustand und berechnet die Commitments erneut. Es prüft außerdem die aktuelle Erklärung des zuständigen Providers. Jede im Vertrag relevante Abweichung verhindert die Publikation. Im lokalen Prototyp bleiben diese Prüfungen und die Publikation innerhalb einer exklusiven Schreibsperre. Der veröffentlichte Inhalt ist Teil derselben atomar sichtbaren Transaktionsdatei.

### 2.2 Begriffe und Bindungsfunktion

| Symbol | Bedeutung                                                                  |
|--------|----------------------------------------------------------------------------|
| L      | Geordnete, validierte Transaktionshistorie                                 |
| S      | Durch deterministischen Replay von L erzeugter Zustand                     |
| M      | Vollständig aufgezähltes Manifest der ausgewählten Implementierungsdateien |
| H      | Digest des typisierten History-Commitments                                 |
| Z      | Digest des typisierten State-Commitments; im Code <code>roots.state</code> |
| I      | Digest des typisierten Implementation-Commitments                          |
| A      | Vollständiger konkreter Aktionskörper                                      |
| Q      | Antrag mit A, H, Z, I, Geltungsbereich, Rolle, Profil, Head und Sequenz    |
| D      | Kanonischer SHA-256-Digest des Antrags Q                                   |

In der Literatur werden sowohl Zustandsobjekte als auch deren Hashwerte häufig mit S bezeichnet. Dieses Paket verwendet Z für den Zustandsdigest, um beide Größen auseinanderzuhalten. Die Implementierung verwendet die Feldnamen `history`, `state` und `implementation`.

Die folgende Darstellung beschreibt die Datenbeziehungen, nicht eine zusätzliche Protokollschicht:

```

S = Replay(L)
H = SHA256(Canonical(HistoryEnvelope(L)))
Z = SHA256(Canonical(StateEnvelope(S)))
I = SHA256(Canonical(ImplementationEnvelope(M)))
D = SHA256(Canonical(Q(A, H, Z, I, Rolle, Profil, Head, Sequenz)))

```

Das Commitment ist eine Bindung an Daten relativ zum verwendeten Hashverfahren. Es ist weder eine Verschlüsselung noch für sich genommen eine digitale Unterschrift oder ein Identitätsnachweis.

## 2.3 Ablauf vom Antrag bis zum Effekt

1. Der Antragsersteller liest unter lokaler Schreibsperre die Historie und prüft ihre Struktur, Reihenfolge und Inhaltsreferenzen.
2. Ein deterministischer Replay erzeugt den aktuellen Zustand. Der Commitment-Baustein berechnet H, Z und I.
3. Der Antragsersteller fixiert Aktion, Inhalt und alle Kontextwerte. Nach der Vorbereitung wird die Sperre freigegeben; die menschliche Prüfung kann zeitlich getrennt stattfinden.
4. Eine benannte Person erklärt ihre Zustimmung zu einem Antragsdigest über den vorgesehenen Provider. Im persönlichen Demonstrationslauf ist dies ein GitHub-Kommentar mit einer ausdrücklichen Selbsterklärung.
5. Der Ausführungsprozess erwirbt erneut die lokale Schreibsperre. Er lädt und prüft den vollständigen aktuellen Kontext sowie den gespeicherten Antrag.
6. Der Prozess liest die Providerdaten frisch ein, prüft ihre Inhalts- und Identitätsbindung und liest den lokalen Kontext anschließend nochmals. Die Sperre bleibt gehalten.
7. Nur wenn alle vorgeschriebenen Prüfungen bestehen, veröffentlicht der Writer eine neue Transaktionsdatei mit dem Testinhalt und den gespeicherten Nachweisen.
8. Die neue Transaktion verändert die Historie. Ein erneuter Versuch mit dem alten Antrag findet deshalb einen abweichenden Kontext und erzeugt keine weitere Publikation.

## 2.4 Technische Wirkung im belegten Modell

**Kontextabweichungen werden vor dem lokalen Effekt erkannt.** Die Versuche zeigen Zurückweisungen nach neuer Evidenz, geänderter Konfiguration, Rollen- und Profilwechseln sowie Änderungen der Test-Providererklärung.

**Historie und definierter Zustand sind getrennt vergleichbar.** Eine Auditnotiz kann H verändern, während Z unverändert bleibt. Vertrag 1 bindet trotzdem strikt an die neue Historie und verwirft den alten Antrag.

**Die geprüfte Aktion entspricht dem gespeicherten Effekt.** Der Aktionsinhalt im Antrag muss dem Inhalt des veröffentlichten Artefakts entsprechen. Der Writer schreibt das Artefakt direkt in die Transaktion; ein späterer ungebundener zweiter Schreibaufruf entfällt für diesen Demonstrationseffekt.

**Konkurrierende lokale Ausführungen werden serialisiert.** Zwei Prozesse verwenden dieselbe Sperre. Der erste erfolgreiche Prozess verändert den Kontext, bevor der zweite seine aktuelle Prüfung abschließen kann. Der Nachweis betrifft höchstens eine Publikationsdatei für denselben Antrag in diesem Ledger.

**Historische Nachweise bleiben bei Korrekturen erhalten.** Im synthetischen Widerrufsversuch wird eine Korrektur angehängt. Die Zustandsprojektion entfernt die aktuelle Wirksamkeit der früheren Publikation, ohne deren historische Datei zu löschen.

## 2.5 Zu untersuchende technische Merkmalskombination

Für die anwaltliche Untersuchung lässt sich der technische Gegenstand in sieben Merkmale zerlegen. Diese Aufstellung ist eine Analysehilfe und kein ausformulierter Patentanspruch.

| Merkmal | Konkretisierung im Prototyp                                                           |
|---------|---------------------------------------------------------------------------------------|
| M1      | Validierte, inhaltsadressierte Historie und deterministischer Replay                  |
| M2      | Unterschiedlich typisierte Commitments für Historie und festgelegten Zustand          |
| M3      | Separates Commitment über ein vollständig aufgezähltes Implementierungsmanifest       |
| M4      | Gemeinsame Bindung von Aktion, Commitments, Rolle, Profil, Head und Sequenz im Antrag |
| M5      | FrISChe Providerprüfung und erneutes Lesen des lokalen Kontexts vor der Publikation   |
| M6      | Gemeinsame lokale Sperre und atomare Veröffentlichung des eigentlichen Testinhalts    |
| M7      | Angehängte Korrektur mit historischer Erhaltung und geänderter aktueller Wirksamkeit  |

Die Hashfunktion, Hashketten, deterministische Serialisierung, Sperren und strukturierte Freigaben sind bekannte Bausteine. Welche Kombination nach dem relevanten Stand der Technik überhaupt abgrenzbar ist, bleibt Gegenstand der Recherche. Die Unterlagen machen keine Aussage, dass sämtliche Merkmale für einen möglichen Anspruch erforderlich, neu oder erfinderisch sind.

## 2.6 Varianten und noch nicht implementierte Ausgestaltungen

Der Mechanismus ist grundsätzlich von GitHub und von einem Governance-Befund als Fachobjekt trennbar. Andere Provider könnten einen Antrag über eine interne Freigabeanwendung, eine Signaturinfrastruktur oder eine dafür zugelassene Maschinenidentität bestätigen. Dafür wäre jeweils ein eigener überprüfter Providervertrag erforderlich.

Ein Datenbank-Adapter könnte dieselbe Bindungsprüfung innerhalb einer Transaktion mit Versionsprüfung ausführen. Für externe Effekte kommen beispielsweise bedingte Zieloperationen, Fencing-Tokens oder eine Outbox mit idempotentem Empfänger in Betracht. Diese Begriffe beschreiben mögliche nächste Ausführungsformen; ihre Korrektheit und Kombination sind im vorliegenden Prototyp nicht nachgewiesen.

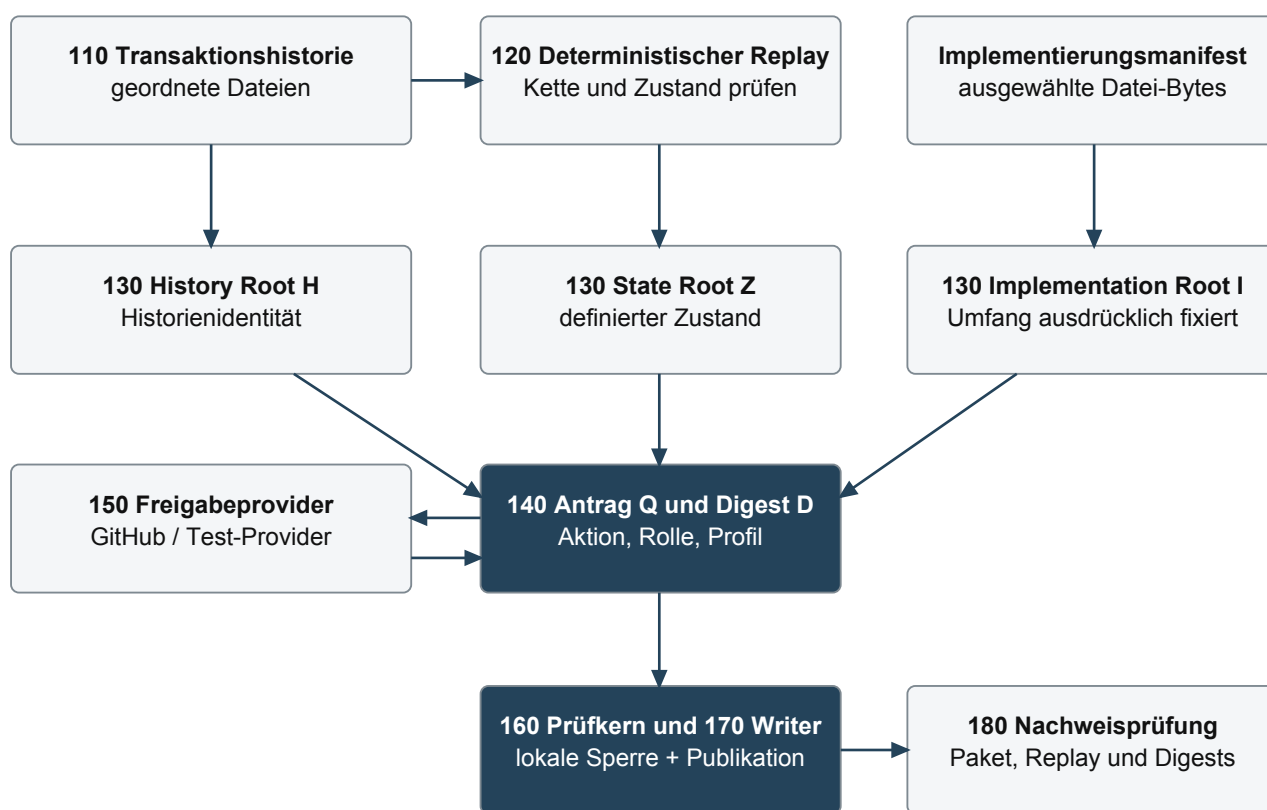
Auch eine selektivere Bindung an Teilzustände wäre eine neue Vertragsversion. Vertrag 1 erlaubt keinen stillen Wechsel auf einen vermeintlich kompatiblen Zustand. Eine aufwendigere Variante darf deshalb nicht als bereits vorhandene Funktion des kleinen Prototyps verstanden werden.

## 3 Design und Architektur

### 3.1 Ausführungsform und Systemgrenze

Die implementierte Ausführungsform verarbeitet einen synthetischen Consumer und einen Befund auf einem lokalen POSIX-Dateisystem. Alle beteiligten lokalen Schreiber müssen dieselbe Sperre beachten. Die einzig zugelassene Publikationsaktion heißt `publish_test_artifact`; sie nimmt eine Zeichenkette von höchstens 4.096 UTF-8-Bytes entgegen. Das Artefakt wird als Inhalt einer unveränderlich behandelten JSON-Transaktion sichtbar. Es wird kein Deployment und kein beliebiger Shell-Befehl ausgeführt.

Das System besteht aus einem lokalen Kern, einem optionalen Adapter für die persönliche GitHub-Erklärung und einem getrennten Nachweispfad. Abbildung 1 zeigt die Komponenten und ihre Datenbeziehungen. Bezugszeichen 110 bis 180 bezeichnen die implementierten Komponenten. Das Diagramm ist eine technische Erläuterung und keine Behauptung einer verteilten atomaren Transaktion.



*Provider und Laufzeitvertrauen liegen außerhalb der lokalen Atomizitätsgarantie.*

Abbildung 1 Komponenten und Bindungsbeziehungen

| Zeichen | Komponente            | Implementierte Verantwortung                                 |
|---------|-----------------------|--------------------------------------------------------------|
| 110     | Transaktionsspeicher  | Geordnete Dateien, Inhaltsidentität, exklusive lokale Sperre |
| 120     | Replay und Projektion | Kette prüfen und definierten Zustand ableiten                |
| 130     | Commitment-Berechnung | H, Z und I aus expliziten Datenstrukturen bilden             |

| Zeichen | Komponente                | Implementierte Verantwortung                                       |
|---------|---------------------------|--------------------------------------------------------------------|
| 140     | Antragsersteller          | Aktionsinhalt und Kontextwerte unveränderlich zusammenstellen      |
| 150     | Freigabeprovider          | Test-Provider oder persönliche Erklärung über GitHub               |
| 160     | Prüf- und Ausführungskern | Aktuellen Kontext, Erklärung und Antrag unter Sperre prüfen        |
| 170     | Atomarer Writer           | Testinhalt und Nachweise als eine neue Transaktion veröffentlichen |
| 180     | Nachweisprüfung           | Aufbewahrte Daten und deklarierte Ergebnisse erneut prüfen         |

### 3.2 Persistente Daten und Zustand

Ein Workspace enthält `EXPERIMENT.json`, `executor-config.json`, `provider.json` und `ledger/transactions/`. Der persönliche Demonstrationslauf ergänzt `personal-request.json` und die Erklärungsvorlage. Die lokale Sperrdatei gehört nicht zur fachlichen Historie. Ein freigegebener Prototypbereich im Repository ist von offiziellen Statusindizes, Website-Ausgaben und Releases getrennt.

Die Historie enthält Transaktionen mit exakt diesen Feldern: `version`, `scope`, `sequence`, `previous_ref`, `event`, `transaction_id`. `previous_ref` enthält ID und Digest der vorangegangenen vollständigen Transaktion. Die neue Transaktions-ID wird aus dem kanonischen Inhalt ohne das Feld `transaction_id` berechnet. Der Dateiname bindet Sequenz und Inhaltsidentität. Replay prüft jede Transaktion und jede Zustandsänderung; ein behaupteter Head allein genügt nicht.

Der Ausgangszustand besteht aus sechs Feldern:

| Zustandsfeld                  | Inhalt und Bedeutung                                                                     |
|-------------------------------|------------------------------------------------------------------------------------------|
| <code>revision</code>         | Semantische Revision; Auditnotizen erhöhen sie nicht                                     |
| <code>latest_evidence</code>  | Letzter akzeptierter synthetischer Prüflauf mit <code>run</code> und <code>result</code> |
| <code>publication</code>      | Aktuell wirksamer Antragsdigest und Artefakt oder <code>null</code>                      |
| <code>role_binding</code>     | Synthetisches Subjekt und Aktivstatus                                                    |
| <code>profile</code>          | Kennung und Version des Versuchsprofils                                                  |
| <code>revoked_requests</code> | Sortierte Liste ausdrücklich widerrufener Antragsdigests                                 |

Die Zustandsprojektion ist absichtlich klein. Sie enthält keine vollständige Governance-Baseline und keine beliebige Unternehmensberechtigung. Ihre Revision gehört ausdrücklich zur Semantik. Zwei Ereignisfolgen, die dieselben fachlichen Statuswerte, aber unterschiedliche semantische Revisionen ergeben, müssen deshalb nicht denselben Z-Wert haben. Nachgewiesen ist die Gleichheit von Z bei einer reinen Auditnotiz.

### 3.3 Kanonisierung und Trennung der Hashdomänen

Alle neuen Commitments nutzen SHA-256 über die vorhandene kanonische JSON-Darstellung des Repositorys. Objektschlüssel werden sortiert; Zeichen werden als ASCII-Escapes ausgegeben; Komma und Doppelpunkt sind die Trennzeichen ohne zusätzliche Leerzeichen; die resultierende Zeichenfolge wird als UTF-8 ohne abschließenden Zeilenumbruch gehasht.

Zugelassen sind `null`, boolesche Werte, ganze Zahlen, Zeichenketten, Listen und Objekte mit Zeichenkettenschlüsseln. Gleitkommazahlen, NaN, doppelte JSON-Schlüssel und nicht unterstützte Typen werden zurückgewiesen. Listenreihenfolge bleibt erhalten; die Menge der widerrufenen Anträge wird vor der Darstellung sortiert. Unicode wird nicht zusätzlich normalisiert. Ein boolescher Wert und eine ganze Zahl sind unterschiedliche Werte.

Die kanonische Darstellung ist ein eigener eingeschränkter Repositoryvertrag. Sie wird nicht als Implementierung von RFC 8785 ausgegeben. RFC 8785 ist eine verwandte Referenz für standardisierte JSON-Kanonisierung; eine Übernahme würde eine ausdrücklich versionierte Kompatibilitätsentscheidung erfordern. [E2]

Jeder Root wird über einen typisierten und versionierten Umschlag berechnet. Die folgenden Ausdrücke benennen die genauen Felder; `scope`, `state` und `files` stehen jeweils für die vollständigen Datenobjekte:

```
HistoryEnvelope = {
  commitment_type: "prototype-history", version: "1", scope,
  transaction_count: Anzahl(L), head_ref: Referenz(letzte Transaktion)
}
StateEnvelope = {
  commitment_type: "prototype-state", version: "1", scope, state
}
ImplementationEnvelope = {
  commitment_type: "prototype-implementation", version: "1", files
}
```

Für die leere Historie ist `head_ref=null` und der Zähler null. Es gibt keinen aktuellen Uhrzeitwert in der Root-Berechnung. Die gesamte validierte Vorgeschichte ist über die rekursiven Vorgängerreferenzen gebunden. Es wird weder eine zweite Historienkette noch ein Merkle-Baum angelegt.

### 3.4 Das Implementierungsmanifest

`source_manifest()` zählt die Python-Dateien unmittelbar unter `experiments/state_binding/`, die importierte Lifecycle-Bibliothek, `result_ledger.py`, die Initialisierungsdatei der Bibliothek sowie die Deklarationen für Validierungsabhängigkeiten und Toolchain auf. `implementation_manifest()` ergänzt die tatsächlichen Bytes von `workspace/executor-config.json`.

Das Manifest bildet jeden relativen Pfad auf den SHA-256-Digest seiner unveränderten Datei-Bytes ab. Hinzugefügte, entfernte oder geänderte Dateien innerhalb dieses festgelegten Umfangs beeinflussen I. Symbolische Links werden zurückgewiesen. Das detaillierte Manifest bleibt zusätzlich zum Root erhalten und ermöglicht eine dateigenaue Diagnose.

Das Manifest umfasst nicht automatisch das Betriebssystem, sämtliche installierten Bibliotheksbytes, dynamisch geladene Maschineninstruktionen oder externe Dienste. Die Bindung an Lock-Dateien dokumentiert Abhängigkeitsdeklarationen; sie attestiert nicht, dass ein kompromittierter Interpreter diese Deklarationen korrekt verwendet. Die Vertrauensgrenze muss bei jeder Übertragung neu festgelegt werden.

### 3.5 Antrag und zwei Ebenen der Demonstrationsfreigabe

Der innere Antrag enthält genau `version`, `scope`, `action`, `roots`, `implementation_manifest`, `subject`, `profile`, `role_binding`, `expected_head` und `expected_sequence`. Der Aktionskörper enthält `kind` und `content`. Die erwartete Sequenz ist die Anzahl der bereits vorhandenen Transaktionen, nicht das Feld `state.revision`.

Der automatische Test-Provider bestätigt den vollständigen Antragsdigest für `test-person:owner`. Seine Erklärung enthält `request_digest`, `subject`, `status`, `environment` und eine Providerrevision. Diese lokale JSON-Struktur ist bewusst ein Test-Double und beweist keine tatsächliche Identität.

Für den persönlichen Lauf gibt es zusätzlich einen äußeren Antrag vom Typ `private-prototype-local-demo`. Er bindet den gesamten inneren Grant, das konkrete private Repository, PR-Nummer, GitHub-Subjekt-ID, Versuchsrolle, Erstellungszeit, Zweck und `official_state=false`. Die reale Person bestätigt den Digest dieses äußeren Antrags. Dadurch wird der tatsächliche lokale Demo-Effekt zusätzlich an ihre Erklärung gebunden; die synthetischen inneren Daten werden dadurch nicht zu realen Prüfbefunden.

Die Versuchsrolle `prototype_reviewer` ist keine produktive Rollenvergabe. Das Verfahren ersetzt insbesondere nicht die separat erforderliche Betriebsabnahme des bestehenden Governance-Piloten.

### 3.6 Vorbereitung und Publikationsprotokoll

Abbildung 2 trennt die drei Zeitabschnitte: konsistente Vorbereitung, persönliche Prüfung ohne gehaltene lokale Sperre und erneute Prüfung mit Publikation unter Sperre.

#### Vorbereitung

**Snapshot und H / Z / I bilden; konkreten Antrag fixieren**  
lokale Sperre während der Vorbereitung

#### Freigabepause

**Benannte Person prüft Antrag und erklärt Zustimmung**  
lokale Sperre ist freigegeben

#### Ausführung

**Sperre erwerben; aktuellen Kontext und Provider prüfen**  
Provider doppelt lesen; gesamten lokalen Kontext erneut prüfen

**Bei Übereinstimmung eine vollständige Transaktion publizieren**  
Artefakt ist Teil derselben lokal atomaren Datei

*Bei Abweichung: keine Publikation. Nach Erfolg: alter Antrag trifft auf neue Historie.*

*Die lokale Sperre sperrt den externen Provider nicht; dessen Race-Fenster bleibt bestehen.*

Abbildung 2 Ablauf und atomare lokale Publikationsgrenze

Der nachfolgende Pseudocode beschreibt den erfolgreichen persönlichen Pfad. Er fasst die tatsächlichen Funktionen zusammen und ist keine zusätzliche ausführbare API:

```
Sperre des lokalen Ledgers erwerben
C1 := Historie laden, replays, Manifest lesen, Roots berechnen
Antrag Q gegen C1 prüfen
Gespeicherte innere Erklärung und aktuellen Test-Provider vergleichen
Ausführungskonfiguration prüfen
GitHub-Antrag und Kommentarbestand direkt erfassen
Kommentarbestand ein zweites Mal lesen und Gleichheit prüfen
Identität, Inhalt, Zeitfolge und Status der Erklärung prüfen
C2 := vollständigen lokalen Kontext nochmals lesen und berechnen
C2 muss C1 entsprechen; Test-Provider muss unverändert sein
Neue Transaktion mit Antrag, Manifest, Erfassung und Artefakt validieren
Transaktion exklusiv veröffentlichen und Verzeichnis synchronisieren
Sperre freigeben
```

`preconditions()` vergleicht Rolle, Aktivstatus, Profil, Widerrufsliste, Vorhandensein von Evidenz, die drei Roots, Head und Transaktionsanzahl. Der normale Pfad korrigiert keine Abweichung automatisch. Die Weglassschalter gehören ausschließlich zu gekennzeichneten Experimenten; der persönliche Pfad gestattet keine ausgelassenen Prüfungen.

Die GitHub-Erfassung prüft die erwartete PR-Adresse und liest die Kommentare zweimal. Der Kommentar muss vom benannten Konto des Typs `User` stammen; eine über eine GitHub-App abgegebene Erklärung wird nicht akzeptiert. Die Erklärung enthält einen festen Marker, den vollständigen Antragsdigest, eine Disposition, eine Vorgängerreferenz und einen ausdrücklichen Selbsterklärungstext. Bearbeitete relevante Kommentare, unpassende Zeitfolgen oder ungültige Vorgängerbeziehungen verhindern eine Bestätigung. Im erfassten Nachweis bleiben die originalen Antwortbytes erhalten.

### 3.7 Lokale Atomizität und Fehlerbehandlung

Der Writer legt eine temporäre Datei im Zielverzeichnis an, schreibt ihren vollständigen Inhalt und ruft `fsync` auf. Anschließend wird sie mit einem exklusiven Hardlink unter dem endgültigen Transaktionsnamen sichtbar gemacht. Eine vorhandene Zielfeile wird nicht überschrieben. Danach wird das Verzeichnis synchronisiert und die temporäre Datei entfernt.

Die beobachtbare Aktion ist die Sichtbarkeit der vollständigen Transaktionsdatei einschließlich Artefakt. Dadurch entsteht innerhalb dieser Systemgrenze kein ungeschützter Abstand zwischen einem gespeicherten Freigabebeleg und einem getrennten Artefaktschreibvorgang. Die Aussage gilt unter den dokumentierten lokalen Dateisystemannahmen und für kooperierende Schreiber.

Schlägt eine Vorbedingung fehl, wird keine Publikation angehängt. Der Versuchsrunner bewahrt Ergebnis und Vorher-/Nachher-Dateien außerhalb des Ledgers auf. Ein Prozessabbruch nach Sichtbarkeit der Datei, aber vor einer erfolgreichen Rückmeldung, kann eine unklare Rückmeldung verursachen. Die bereits sichtbare Transaktion bleibt anhand ihres Namens und Inhalts feststellbar. Der alte Antrag erzeugt beim Wiederholungsversuch keinen zweiten Effekt; der Prototyp verspricht keine verteilte Exactly-once-Zustellung oder automatische Wiederholung einer verlorenen Antwort.

### 3.8 Widerruf und aktuelle Wirksamkeit

Im synthetischen Widerrufspfad bestätigt der Test-Provider den Widerruf. `revoke()` hängt eine Korrektur mit dem ursprünglichen Antragsdigest, dem bezeichneten Subjekt und der Testumgebung an. Diese Korrektur benötigt nicht den alten, inzwischen überholten History-Root. Der Replay ergänzt die Widerrufsliste und entfernt eine passende aktuelle Publikation aus der wirksamen Zustandsprojektion. Die frühere Transaktion bleibt erhalten.

Der Prototyp hat keine automatische Überwachung späterer GitHub-Kommentaränderungen. Insbesondere ist der gesamte Weg von einem späteren realen GitHub-Widerruf zu einer automatisch angehängten lokalen Korrektur nicht implementiert. Der persönliche Nachweis bestätigt die Erklärung zu den dokumentierten Erfassungszeitpunkten. Er darf nicht mit einer fortlaufenden Aussage über deren heutigen Status verwechselt werden.

### 3.9 Vertrauensmodell und verbleibende Zeitfenster

Die lokale Sperre schützt den Ledger gegen andere kooperierende lokale Writer. Sie sperrt nicht GitHub. Eine Providererklärung kann nach der letzten erfolgreichen Abfrage und vor dem lokalen Hardlink geändert oder widerrufen werden. Der Prototyp verkleinert und dokumentiert das Prüfzeitfenster; er beseitigt diese verteilte Race Condition nicht. Eine stärkere Garantie erfordert einen zusätzlichen Providervertrag oder ein gemeinsam kontrolliertes Autorisierungs- und Commit-Protokoll.

Entsprechend schützt die Ledgersperre nicht gegen einen privilegierten Angreifer, der Implementierungsdateien oder Laufzeitspeicher außerhalb des Protokolls verändert. Das Verfahren setzt einen vertrauenswürdigen Ausführungskern, Interpreter, Dateisystem und Providerzugriff voraus. Die erneute vollständige Kontextprüfung erkennt beobachtbare Änderungen während der Providerabfrage, garantiert jedoch keine Hardware-Attestierung.

Ein selbstkonsistent neu geschriebenes oder gekürztes Ledger kann mit neuen Hashwerten wieder intern konsistent erscheinen. Manipulationserkennung benötigt deshalb einen vertrauenswürdigen aufbewahrten früheren Head, Präfix oder Paketdigest. Ebenso gilt: Ein unveränderter Root beweist weder Aktualität der fachlichen Evidenz noch einen Ablaufzeitpunkt. Vertrag 1 besitzt keinen TTL- oder Freshness-Vertrag; zeitabhängige Gültigkeit müsste gesondert spezifiziert werden.

### 3.10 Kompatibilität und Erweiterungsgrenzen

Der Forschungsbereich verwendet ausschließlich seinen eigenen Vertrag 1. Er ändert keine akzeptierte produktive Transaktion, kein offizielles Evidenzschema, keine OPA-Policy und keine freigegebene Baseline. Die 83 durch vorhandene Betriebsabnahmen gebundenen Implementierungsdateien sind gegenüber dem dokumentierten Ausgangsstand unverändert.

Eine spätere Übertragung benötigt mindestens einen definierten Projektionsvertrag für das neue Fachobjekt, einen vollständig beschriebenen Implementierungsumfang, einen zulässigen Provider und eine belastbare Kopplung zum Zielschreibvorgang. Für mehrere unabhängige Systeme reicht die hier verwendete lokale Sperre nicht aus. Eine Integration in Bitbucket/Bamboo wäre ein neuer Adapter mit eigenem Identitäts-, Capture- und Ausführungsnachweis; sie ist nicht Bestandteil der vorliegenden Implementierung.

## 4 Anwendungen außerhalb der Repository Governance

### 4.1 Gemeinsames Anwendungsprinzip

Die Lösung ist dort technisch interessant, wo zwischen der Vorbereitung einer zustimmungspflichtigen Aktion und ihrem tatsächlichen Effekt relevante Veränderungen auftreten können. Voraussetzung ist ein bestimmbarer Kontext, eine definierte Zustandsprojektion und ein kontrollierter Ausführungspfad. Die folgenden Anwendungsformen sind technische Übertragungsentwürfe. Mit Ausnahme des lokalen Governance-Demonstrators sind sie noch nicht implementiert oder experimentell nachgewiesen.

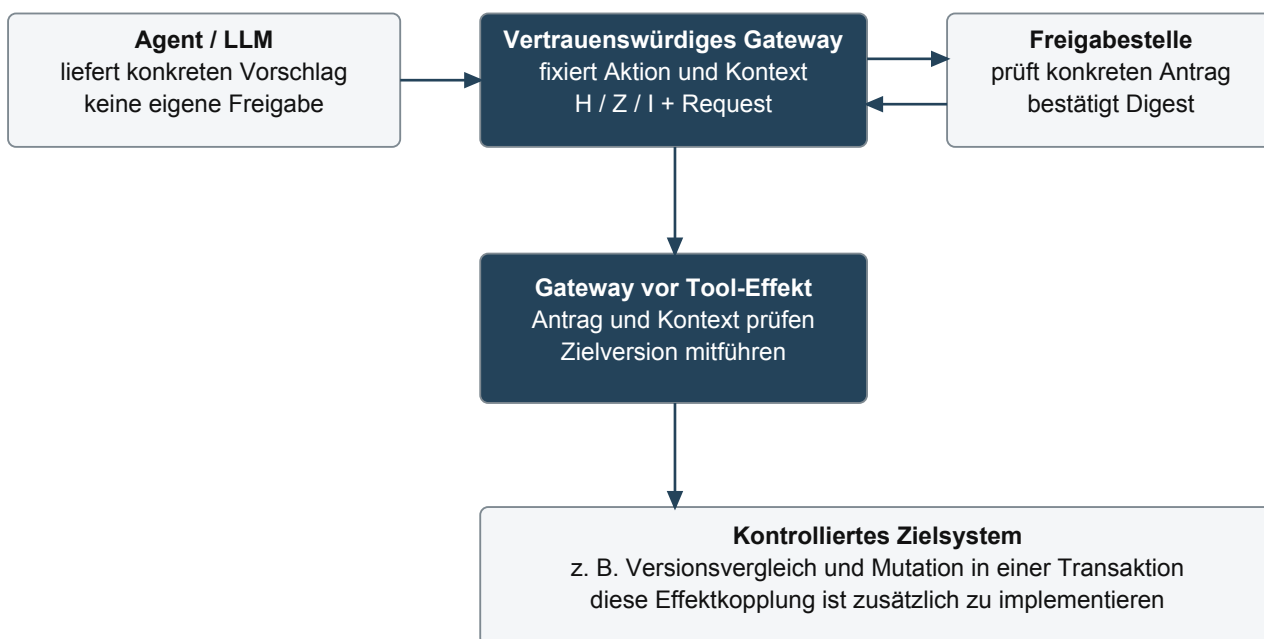
| Anwendungsfeld                     | Mögliche gebundene Daten                                                            | Erwarteter Nutzen und zusätzlicher Bedarf                                                                   |
|------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| KI-Agent mit Werkzeugzugriff       | Konkreter Toolaufruf, Zielobjektversion, akzeptierter Verlauf, Adapterkonfiguration | Veraltete Zustimmung zum Toolaufruf erkennen; Zielsystem muss bedingten Effekt unterstützen                 |
| Zusammenarbeit mehrerer Agenten    | Auftragszustand, delegierte Rolle, freigegebener Teilplan, Implementierungsstand    | Kontextwechsel zwischen Agenten sichtbar machen; Identität und Delegation zusätzlich regeln                 |
| Freigabe eines KI-Modells          | Modelldigest, Evaluationsnachweise, Einsatzprofil, Serving-Konfiguration            | Release an genau die geprüfte Kombination binden; echte Deployment-Kopplung ergänzen                        |
| Infrastrukturänderung              | Konkreter Änderungsplan, Ressourcenstände, Ausführungskonfiguration                 | Abweichung zwischen geprüftem Plan und Zielzustand erkennen; Cloud-/Cluster-Adapter erforderlich            |
| Datenmigration                     | Schemafassung, freigegebene Transformation, Dataset-Version                         | Freigabe für eine andere Datenbasis zurückweisen; Transaktion oder Snapshot im Ziel erforderlich            |
| Automatisierte Sicherheitsreaktion | Befundstand, betroffene Ressource, ausgewählte Reaktion, Richtlinienversion         | Zustimmung an aktuelle Lage binden; Nebenwirkungen und Rücknahme eigenständig modellieren                   |
| Produktionssteuerung               | Version eines Arbeitsauftrags, Maschinenkonfiguration, zulässiger Parametersatz     | Einsatz überholter Parametergenehmigung erkennen; Echtzeit- und Sicherheitsanforderungen separat nachweisen |

### 4.2 KI-Agent mit zustimmungspflichtigem Werkzeugaufruf

Ein Agent schlägt beispielsweise eine Änderung an einem Kundendatensatz vor. Die Person sieht den konkreten Zielschlüssel, die bisherigen Werte und den vorgeschlagenen Patch. Während der Freigabepause verändert ein anderer Prozess den Datensatz oder der Agent überarbeitet seinen Plan. Eine unveränderte allgemeine Zustimmung zum Vorgang darf dann nicht automatisch den neuen konkreten Schreibaufruf autorisieren.

Für eine Übertragung würde der Agent zunächst nur einen typisierten Vorschlag erzeugen. Ein vertrauenswürdiges Gateway bildet den konkreten Aktionskörper A, nimmt die Objektversion in den relevanten Zustand auf und berechnet den gebundenen Antrag. Nach der Zustimmung prüft das Gateway den Kontext erneut und führt ausschließlich die gebundene Operation aus. Es übergibt dem Zielsystem beispielsweise eine erwartete Version für eine bedingte Änderung. Ohne eine solche Zielbedingung bleibt zwischen lokaler Prüfung und externem Effekt ein offenes Zeitfenster.

### Entwurf einer Übertragung auf Agentensysteme - noch nicht implementiert



*LLM-Ausgaben werden aufgezeichnet; ein deterministischer Replay ruft das Modell nicht neu auf.*

Abbildung 3 Übertragung auf einen Agenten mit kontrolliertem Werkzeugzugriff

Die Grenze zwischen Agent und Gateway ist wesentlich: Das Sprachmodell darf den finalen Antrag nicht nachträglich austauschen, seine eigene Freigabe erzeugen oder den kontrollierten Adapter umgehen. Die Plattform muss dafür Zugangsdaten und direkte Werkzeugrechte entsprechend kapseln. Der Prototyp implementiert eine solche Agentenplattform noch nicht.

Nichtdeterministische LLM-Ausgaben werden für einen Replay als aufgezeichnete Eingaben oder konkrete ausgewählte Aktionen behandelt. Der Replay darf nicht voraussetzen, dass ein erneuter Modellaufruf dieselbe Antwort erzeugt. Ein Modellname allein bindet zudem kein unveränderliches Modellartefakt, wenn ein externer Anbieter unter demselben Namen die Implementierung ändern kann.

Die Kontextbindung bewertet weder die inhaltliche Qualität eines Modellvorschlags noch verhindert sie jede Prompt Injection. Eine fehlerhafte, aber unverändert freigegebene Aktion kann weiterhin fehlerhaft sein. Sie unterstützt die technische Trennung von Vorschlag, Zustimmung und tatsächlich zugelassenem Effekt.

### 4.3 Mehrere kooperierende Agenten

Ein planender Agent könnte eine Aufgabe zerlegen, ein zweiter einen Patch erzeugen und ein dritter Testergebnisse bewerten. Eine menschliche Freigabe müsste dann die tatsächlich zur Ausführung ausgewählte Variante und den zugehörigen Zustand binden. Neue Testergebnisse oder ein Ersatzpatch müssten den alten Antrag ungültig machen, soweit sie zum gebundenen Kontext gehören.

Diese Ausführungsform erfordert zusätzlich ein überprüfbares Delegationsmodell. Der bisherige feste synthetische Owner reicht dafür nicht. Agentenidentitäten, Auftragsscope, delegierte Befugnisse und die maximale Wirkung jedes Teilauftrags müssten ausdrücklich modelliert werden. Mehrere Agenten benötigen nicht zwingend mehrere Ledger; ihre Ausführung könnte zunächst über ein gemeinsames vertrauenswürdigen Gateway serialisiert werden. Ein verteilter Ausbau ist eine weitere Architekturentscheidung.

### 4.4 KI-Modellfreigabe und MLOps

Für einen Modellrelease könnte die Zustandsprojektion die geprüfte Modellversion, Evaluationsdatensätze, relevante Testergebnisse und das freigegebene Einsatzprofil enthalten. I könnte den tatsächlichen Serving-Adapter, die Vor-/Nachverarbeitung und eine festgelegte Konfiguration binden. Die konkrete Aktion wäre beispielsweise die Aktivierung genau eines bestimmten Modellartefakts in einem benannten Ziel.

Eine solche Bindung wäre hilfreich, wenn nach der Prüfung neue Evaluationsergebnisse eintreffen oder eine andere Vorverarbeitung ausgerollt wird. Sie ist jedoch kein Beweis für Modellqualität, Fairness oder Sicherheit. Die reale Modellbereitstellung müsste bedingt und nachvollziehbar mit der Freigabe gekoppelt werden; der lokale Artefaktbeschreibvorgang des Prototyps genügt dafür allein nicht.

### 4.5 Planbasierte Änderungen und Datenverarbeitung

Bei Infrastruktur-as-Code oder einer Datenmigration besteht ein ähnliches Muster: Geprüft wird ein konkreter Plan für einen bestimmten Ausgangszustand. Die spätere Ausführung muss feststellen, ob Plan und Ziel weiterhin zusammenpassen. Die H-/Z-/I-Bindung kann die erwartete Grundlage explizit machen. Eine Datenbank könnte den Versionsvergleich und die Mutation in derselben Transaktion ausführen; bei einem Cloud-API-Aufruf wären bedingte Updates oder ein eigens entworfenes Ausführungsprotokoll erforderlich.

### 4.6 Eignung und Grenzen der Übertragung

Die Bindung ist besonders sinnvoll bei zustandsabhängigen, prüfbaren und kontrolliert auslösbaren Operationen. Weniger geeignet ist der unveränderte Prototyp für unkontrollierbare externe Effekte, harte Echtzeitvorgaben oder Akteure, die den gemeinsamen Ausführungspfad umgehen dürfen.

Bereits vorhandene Agentenframeworks kennen Freigabeunterbrechungen und Wiederaufnahme. LangGraph dokumentiert beispielsweise Unterbrechungen für menschliche Eingaben und die Bedeutung idempotenter Seiteneffekte bei erneut ausgeführten Knoten. Diese Funktionen sind als bekannte Vergleichsmechanismen zu berücksichtigen. Daraus folgt weder eine Integration des vorliegenden Prototyps in LangGraph noch eine Aussage, dass andere Frameworks die gesamte hier beschriebene Merkmalskombination besitzen oder nicht besitzen. [E5]

## 5 Implementierungsreferenzen und Nachweise

### 5.1 Was tatsächlich implementiert ist

Die hier beschriebene vollständige lokale Ausführungsform ist im privaten Forschungsbereich `experiments/state_binding/` implementiert. Referenzstand dieser Unterlage ist Commit `56ed0c743121fddc9d117b69eb470493dff5046d` im Repository `joku-dev/devsecops-governance-framework`. Dieser Commit enthält den persönlichen Demonstrationsnachweis. Die gebundenen Python-Quelldateien sind gegenüber dem ursprünglichen Prototyp-Commit unverändert.

Es wird keine weitere Installation derselben vollständigen Lösung bei Dritten behauptet. Das Consumer-Repository `ha-CPsWMS` ist ein Ziel des bestehenden Governance-Frameworks, aber kein Nachweis einer dort bereits integrierten H-/Z-/I-Prototypausführung. Die KI- und Agentenanwendungen in Kapitel 4 sind ebenfalls Übertragungsentwürfe.

Das Weitergabepaket enthält einen gezielten Quellcodeauszug mit den gebundenen Implementierungsdateien und den Prototyptests sowie die originalen Evidence-Pakete. Der Auszug ist zur Offline-Inspektion bestimmt. Er ersetzt nicht den vollständigen Repository-Checkout für die Gesamttests. Der eigenständige Prüfer des automatisierten Evidence-Pakets benötigt lediglich die Python-Standardbibliothek.

### 5.2 Feste Codefundstellen

Die Links in dieser Tabelle sind auf den oben genannten Commit festgelegt und benötigen Zugriff auf das private Repository. Alle angegebenen Python-Dateien sind außerdem im Quellcodeauszug enthalten.

| Funktion                             | Implementierte Stelle                                                                                                      | Bezugsmerkmale           |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------|
| Historie und Zustand getrennt binden | <a href="#">prototype.py Zeile 102</a> , <code>history_payload</code> , <code>state_payload</code> , <code>roots</code>    | M1, M2                   |
| Implementierungsmanifest und Root    | <a href="#">prototype.py Zeile 114</a> , <code>source_manifest</code> , <code>implementation_manifest</code>               | M3                       |
| Antrag und Vorbedingungen            | <a href="#">prototype.py Zeile 144</a> , <code>validate_request</code> , <code>preconditions</code> , <code>prepare</code> | M4                       |
| Replay und Zustandsübergänge         | <a href="#">prototype.py Zeile 179</a> , <code>reduce_event</code> , <code>replay</code>                                   | M1, M7                   |
| Erneute Prüfung und Publikation      | <a href="#">prototype.py Zeile 338</a> , <code>execute</code>                                                              | M5, M6                   |
| Angehängter synthetischer Widerruf   | <a href="#">prototype.py Zeile 370</a> , <code>revoke</code>                                                               | M7                       |
| Lokale Sperre und atomarer Writer    | <a href="#">store.py Zeile 14</a> , <code>writer_lock</code> , <code>publish_transaction</code>                            | M6; bestehender Baustein |
| Persönlicher Demo-Adapter            | <a href="#">human_demo.py</a> , <code>prepare</code> , <code>execute</code>                                                | M4, M5                   |

| Funktion                             | Implementierte Stelle                                                                                    | Bezugsmerkmale              |
|--------------------------------------|----------------------------------------------------------------------------------------------------------|-----------------------------|
| Prüfung der GitHub-Erklärung         | <code>personal_probe.py</code> Zeile 42, <code>assess_bound_comments</code> , <code>collect_bound</code> | M5; bestehender Baustein    |
| Versuche und eigener Evidence-Prüfer | <code>run_experiments.py</code> und <code>verify_evidence.py</code>                                      | Experimentelle Überprüfung  |
| Automatische Testfälle               | <code>test_state_binding_prototype.py</code>                                                             | 17 zusätzliche Testmethoden |

### 5.3 Aufbewahrte Versuche

Der automatisierte Lauf umfasst 18 tatsächliche lokale Dateisystemversuche. Der Runner prüft nicht nur Rückgabewerte, sondern auch die Anzahl und den Inhalt der Transaktionen vor und nach dem jeweiligen Versuch. Der Konkurrenzversuch verwendet zwei verschiedene Prozesse. Alle automatisierten Providererklärungen sind synthetisch und als solche ausgewiesen.

| Versuch                                            | Ergebnis       | Ausgelassene Prüfungen  | Status |
|----------------------------------------------------|----------------|-------------------------|--------|
| <code>unchanged</code>                             | Publiziert     | Keine                   | PASS   |
| <code>new_evidence</code>                          | Zurückgewiesen | Keine                   | PASS   |
| <code>audit_only_history</code>                    | Zurückgewiesen | Keine                   | PASS   |
| <code>implementation_changed</code>                | Zurückgewiesen | Keine                   | PASS   |
| <code>role_changed</code>                          | Zurückgewiesen | Keine                   | PASS   |
| <code>profile_changed</code>                       | Zurückgewiesen | Keine                   | PASS   |
| <code>action_changed</code>                        | Zurückgewiesen | Keine                   | PASS   |
| <code>provider_revoked</code>                      | Zurückgewiesen | Keine                   | PASS   |
| <code>provider_edited</code>                       | Zurückgewiesen | Keine                   | PASS   |
| <code>provider_deleted</code>                      | Zurückgewiesen | Keine                   | PASS   |
| <code>state_root_changed</code>                    | Zurückgewiesen | Keine                   | PASS   |
| <code>implementation_without_binding</code>        | Publiziert     | implementation          | PASS   |
| <code>history_without_history_root</code>          | Zurückgewiesen | history                 | PASS   |
| <code>history_without_history_preconditions</code> | Publiziert     | history, head, revision | PASS   |
| <code>state_without_state_root</code>              | Publiziert     | state                   | PASS   |

| Versuch                              | Ergebnis                               | Ausgelassene Prüfungen | Status |
|--------------------------------------|----------------------------------------|------------------------|--------|
| <code>retry</code>                   | 1 Publikation; Wiederholung abgewiesen | Keine                  | PASS   |
| <code>revocation_after_change</code> | Widerruf wirksam; Historie erhalten    | Keine                  | PASS   |
| <code>concurrent_publication</code>  | 2 Prozesse; 1 Publikation              | Keine                  | PASS   |

Das Ergebnis ist **18 von 18 erwartungsgemäß bestandenen Versuchen**. Eine erwartete Zurückweisung zählt als bestandener Negativtest. Eine im Weglassversuch absichtlich zugelassene Publikation ist ebenfalls ein erwartetes Experimentergebnis; sie ist keine produktive Freigabe und kein versehentlich akzeptierter Sicherheitsbefund.

Zusätzlich sind 27 unveränderte Tests des vorhandenen Pilot-Aktions- und Betriebsabnahmecodes als Vergleich ausgeführt worden. Sie sind kein identischer End-to-End-Benchmark zweier austauschbarer Produkte. Die vollständige Repository-Validierung nach dem persönlichen Lauf bestand mit 640 Tests. OPA, Runtime-, Repository- und Provenance-Prüfungen waren erfolgreich. Die Prototyptests sind in diesen 640 Tests enthalten, nicht zusätzlich zu addieren.

## 5.4 Erkenntnisse aus Weglassversuchen

Wird ausschließlich die zusätzliche History-Root-Prüfung weggelassen, verhindert die bestehende Head-/Sequenzprüfung weiterhin die Publikation auf einer geänderten Historie. Erst das Weglassen aller drei Historienvorbildungen im betreffenden Versuch lässt die geänderte Historie bei unverändertem definierten Zustand zu. Dies belegt eine Überschneidung vorhandener Schutzmechanismen.

Wird die Implementierungsbindung weggelassen, kann eine nach der Freigabe geänderte Ausführungskonfiguration verwendet werden. Die normale Bindungsprüfung weist denselben Fall zurück. Eine weitere Versuchspaarung prüft einen bewusst falsch angegebenen State-Root bei ansonsten passenden Daten. Diese Versuche zeigen den Einfluss der jeweiligen Prüfung, begründen aber keinen automatischen Neuheits- oder Überlegenheitsnachweis gegenüber anderen korrekt implementierten Verfahren.

## 5.5 Persönlicher Demonstrationslauf

Am 20. September 2026 gab das benannte Konto `joku-dev`, GitHub-ID `81616324`, die antragsgebundene Erklärung selbst in [PR-Kommentar 5749239832](#) ab. Der Ausführungsprozess erfasste und prüfte sie um `10:33:43 UTC`. Er veröffentlichte genau ein lokales Artefakt mit dem Inhalt `Personally authorized PRIVATE LOCAL TEST artifact`.

Die Erfassung blieb zusammen mit dem Antrag und dem Artefakt in der Transaktion erhalten. Eine erneute direkte Providerabfrage um `10:35:21 UTC` bestätigte dieselbe Erklärung. Die nachfolgende Offline-Prüfung rekonstruierte Kette und Zustand. Eine rein lesende Wiederverwendungsprüfung wies den alten Antrag wegen `stale_history` zurück; es wurde kein zweiter persönlicher Ausführungsversuch gestartet.

Der personenbezogene Nachweis umfasst die vom Provider bestätigte Kontoidentität und die ausdrückliche Selbsterklärung. Er ist keine Hardware- oder Anwesenheitsattestierung. Die Beobachtung

und innere Fixture-Freigabe bleiben synthetisch. Die GitHub-Erfassung ist eine zeitbezogene Beobachtung und keine permanente Zusage über den späteren Zustand des Kommentars.

| Kennung                                   | Wert                                                             |
|-------------------------------------------|------------------------------------------------------------------|
| Äußerer Antragsdigest                     | 48a97c11dd0fe4f74806f95d2fcd2af019b097d3e129f0a18a9774654ec7ce88 |
| Publikation ohne Präfix<br>transaction:   | b7d3992989486569612bf055cd04af5b25e9d01a8ee8dd776baf79ff82796ed4 |
| Code-Stand der persönlichen<br>Ausführung | 4dbc458872d39c113d8f0b978aab918eb0c6ce28                         |

## 5.6 Integrität und Wiederholbarkeit

| Datei                 | SHA-256 des unveränderten Pakets                                 |
|-----------------------|------------------------------------------------------------------|
| run-001.zip           | 919e299760d1f62d4345ff7a239ef7919c82d23c2c36732d9b8a1d7b665f29ea |
| personal-demo-001.zip | 2183f4a4601b5f96cadd2a649c0ca601ea3f16d4e5305dd5fa47a8c1bc12a38d |

Der eigenständige Prüfer des automatisierten Pakets importiert die Prototypimplementierung nicht. Er rekonstruiert Historie, Zustandsprojektion, Roots und zulässige Effekte selbst. Das persönliche Paket wird dagegen mit dem vorhandenen Replay- und Providerprüfcode geprüft. Diese unterschiedliche Unabhängigkeit ist bei der Bewertung der Beweiskraft zu berücksichtigen.

Nach Entpacken des Weitergabepakets lässt sich der automatische Nachweis ohne GitHub-Zugang und ohne neue persönliche Erklärung prüfen:

```
python3 source/experiments/state_binding/verify_evidence.py \
  evidence/run-001.zip \
  --expect-manifest-sha256 \
  e8a903751e3e3f040371329f8f8ef2910376f72dec42a033f3620b50caf4628
```

Eine Wiederholung der vollständigen Versuche erfolgt im vollständigen Repository-Checkout gemäß `experiments/state_binding/README.md`. Das Nachvollziehen gespeicherter Daten erfordert keine neue Erklärung. Eine weitere tatsächliche persönliche Testaktion benötigt einen neuen Antrag und die dazu passende Erklärung.

Dateihashes und signierte Entwicklungscommits binden Bytes an einen dokumentierten Bezugspunkt. Sie ersetzen keine amtliche Zeitbestätigung und beweisen nicht den Zeitpunkt einer Erfindung. Das zusätzliche `MANIFEST.json` des Weitergabepakets erlaubt die Prüfung aller enthaltenen Dateien.

## 5.7 Hosted CI und produktiver Reifegrad

Die dokumentierte letzte vollständig ausgewertete PR-Revision `4dbc458` bestand Governance CI und Repository Security. Dependency Review war in der Repositorykonfiguration nicht verfügbar; der Consumer Lifecycle Guard erhielt HTTP 404 für eine bestehende Providerdiskussion; CodeQL scheiterte am Zugriff auf die Workflow-Run-API. Die Detailreferenzen liegen in `evidence/CI-LIMITATIONS.md`.

Der private PR #176 ist ein Draft. Der lokale Funktionsnachweis ist abgeschlossen; daraus wird keine Aussage abgeleitet, dass alle Hosted-Checks grün sind oder eine produktive Einführung freigegeben ist. Spätere PR-Zustände sind von diesem festgehaltenen Nachweisstand zu unterscheiden.

## 5.8 Externe technische Referenzen und Vergleichsbausteine

Die folgenden öffentlichen Primärquellen wurden am 20. September 2026 geprüft. Sie erläutern bekannte Problemklassen und verwandte Mechanismen. Die Auswahl ist keine vollständige Patentrecherche. Die Aussagen über den vorliegenden Prototyp stützen sich auf dessen Code und Versuche; die externen Referenzen belegen keine identische Drittimplementierung der gesamten Lösung.

**[E1] MITRE CWE 367.** Beschreibt Zustandsänderungen zwischen Prüfung und Nutzung einer Ressource sowie die Bedeutung geeigneter Synchronisation. Bezug: bekannte TOCTOU-Problemklasse. <https://cwe.mitre.org/data/definitions/367.html>

**[E2] RFC 8785 JSON Canonicalization Scheme.** Beschreibt eine standardisierte kanonische JSON-Darstellung für wiederholbare kryptografische Operationen. Bezug: bekannte Kanonisierung. Der Prototyp verwendet einen eigenen eingeschränkten Vertrag. <https://www.rfc-editor.org/rfc/rfc8785>

**[E3] RFC 9162 Certificate Transparency Version 2.0.** Beschreibt Merkle-Bäume sowie Inklusions- und Konsistenznachweise für Logs. Bezug: bekannte kryptografische Historienbindung. Der Prototyp verwendet eine Hashkette und implementiert kein Certificate-Transparency-Log. <https://www.rfc-editor.org/rfc/rfc9162>

**[E4] in-toto.** Dokumentiert ein implementiertes Framework zur Prüfung von Softwarelieferketten mit definierten Schritten, autorisierten Beteiligten und Artefaktbeziehungen. Bezug: bekannte Bindung technischer Artefakte und Prozessnachweise. Die vollständige lokale H-/Z-/I-Freigabekombination dieses Pakets wird daraus nicht abgeleitet. <https://in-toto.io/docs/what-is-in-toto/>

**[E5] LangGraph Interrupts.** Dokumentiert ein implementiertes Unterbrechungs- und Wiederaufnahmeframework für menschliche Eingaben sowie Anforderungen an Seiteneffekte beim erneuten Ausführen von Knoten. Bezug: bekannte Freigabepausen in Agentenabläufen. Es besteht keine implementierte Anbindung an diesen Prototyp. <https://docs.langchain.com/oss/python/langgraph/interrupts>

**[E6] RFC 9396 OAuth 2.0 Rich Authorization Requests.** Beschreibt strukturierte `authorization_details` für differenzierte Autorisierungsanfragen. Bezug: bekannte explizite Beschreibung erlaubter Aktionen und Ressourcen. Der Prototyp implementiert weder OAuth noch diesen RFC. <https://www.rfc-editor.org/rfc/rfc9396>

Eine Neuheitsbewertung müsste konkrete Veröffentlichungen und deren Zeitpunkte mit den einzelnen technischen Merkmalen vergleichen. Insbesondere dürfen Hashketten, explizite Aktionsfreigaben, Versionsvergleiche, Implementierungsmanifeste und atomare Schreibverfahren nicht allein aufgrund anderer Bezeichnungen als neue Verfahren behandelt werden.

## 6 Herkunft und Fragen für die anwaltliche Prüfung

### 6.1 Zweck und Prüfauftrag

Dieses Paket erläutert die technische Problemstellung, eine konkrete ausführbare Lösung, ihre Systemgrenzen und die experimentellen Nachweise. Es soll die anwaltliche Beurteilung erleichtern, welche technischen Merkmale weiter untersucht werden sollen und welche zusätzlichen Informationen für eine mögliche Anmeldung erforderlich sind. Patentansprüche, Anmelder- und Erfinderbenennung sowie eine rechtliche Bewertung sind nicht Bestandteil des Pakets.

Die DPMA-Hinweise verlangen für eine Anmeldung eine hinreichend deutliche und vollständige technische Offenbarung und nennen unter anderem technische Beschreibung, Ansprüche und gegebenenfalls Zeichnungen als Bestandteile. Das vorliegende Paket ist dafür eine technische Arbeitsgrundlage; die Auswahl und endgültige Fassung der Anmeldeunterlagen obliegt der Beratung. [E7]

### 6.2 Dokumentierte Herkunft und Versionsfolge

Der Maintainer stellte das Arbeitsdokument `WP_STATE_COMMITMENT_AUTHORIZATION_BINDING.md` zur Verfügung und beauftragte anschließend einen begrenzten privaten Prototyp mit technischen Nachweisen. Das Arbeitsdokument trägt das Datum 19. September 2026. Dieses aufgedruckte Datum beweist für sich genommen keinen Entstehungszeitpunkt. Die Originaldatei bleibt unverändert im lokalen Arbeitsverzeichnis und ist nicht Teil des Quellcodeauszugs.

| Dokumentierter Stand                         | Bedeutung                                                                                    |
|----------------------------------------------|----------------------------------------------------------------------------------------------|
| 68849a1011097e9ad9858441<br>10b12c2dd7f18e3b | Prototypcode, Tests und ursprüngliches Design; Quellstand des automatisierten Versuchspakets |
| 6d1a3bae5527223fb5af5467<br>7c902cbba07009f6 | Aufbewahrung automatisierter Versuche und Validierungsnachweise                              |
| 4dbc458872d39c113d8f0b97<br>8aab918eb0c6ce28 | Vorbereiteter persönlicher Antrag; Quellstand der persönlichen Ausführung                    |
| 56ed0c743121fddc9d117b69<br>eb470493dff5046d | Aufbewahrung des abgeschlossenen persönlichen Demonstrationsnachweises                       |

Der Agent hat Implementierung, Experimente und Erläuterungen auf Grundlage des Auftrags vorbereitet. Daraus wird keine rechtliche Zuordnung von Erfinderschaft oder Eigentum abgeleitet. Die vollständigen menschlichen Beiträge, deren Zeitpunkte und etwaige Rechtebindungen müssen vom Maintainer und den beteiligten Personen bestätigt werden.

### 6.3 Technische Fragen an die Beratung

- Welche Kombination aus M1 bis M7 beschreibt einen hinreichend konkreten technischen Gegenstand und welche Merkmale sind bereits in relevanten Veröffentlichungen verbunden?
- Welche Bedeutung hat die nachgewiesene Redundanz zwischen zusätzlichem History-Root und vorhandener Head-/Sequenzprüfung für eine mögliche Abgrenzung?
- Ist der getrennte State-Root in einer genau definierten Ausgestaltung technisch tragend, obwohl der Zustand deterministisch aus bereits gebundenen Eingaben abgeleitet wird?

- Welche Rolle spielt die Implementierungsbindung zusammen mit der erneuten Prüfung an der tatsächlichen lokalen Schreibgrenze?
- Soll eine mögliche Beschreibung auf die nachgewiesene lokale Ausführungsform begrenzt bleiben oder zusätzliche ausreichend ausgearbeitete Ausführungsformen enthalten?
- Welche weiteren Versuche wären für externe Effekte, einen realen späteren Widerruf oder eine konkrete Agentenintegration erforderlich?

## 6.4 Vom Maintainer zu ergänzende Angaben

Für die Beratung sind die vollständigen Namen und konkreten technischen Beiträge der beteiligten Menschen, die frühesten belastbaren Entstehungsunterlagen und mögliche arbeitsvertragliche oder sonstige Rechtebindungen noch zu klären. Das Paket füllt diese Angaben nicht stellvertretend aus.

Ebenfalls offen ist eine vollständige Übersicht früherer Offenlegungen. Zu prüfen sind frühere öffentliche Repositorystände, Releases, GitHub Pages, Vorträge, Präsentationen, Consumer-Integrationen sowie Weitergaben des Arbeitsdokuments. Der während der Prototypentwicklung bestätigte private Repositorystatus belegt nicht, dass keine relevante ältere Veröffentlichung existiert. Gesondert zu erfassen sind Inhalt, Datum, Empfänger und gegebenenfalls vereinbarte Vertraulichkeit einer Weitergabe.

Das dokumentierte Konzept und der Prototyp sollen als technische Nachweise erhalten bleiben. Eine Auswahl passender Länder, Fristen oder Anmeldeverfahren wird hier nicht vorweggenommen. Der Benutzer kann diese Fragen anhand des Pakets direkt mit seinem Patentanwalt klären.

## 6.5 Weitergabe und Vertraulichkeit

Die Dateien sind als vertraulich gekennzeichnet und bleiben außerhalb der Website-, Viewer- und Release-Verzeichnisse. Der Docs-Publisher des Repositorys wurde deaktiviert; ein zusätzlicher Confidential-Marker sperrt die Veröffentlichung dieses Forschungsbereichs. Die Prüfung der Sichtbarkeit ist eine Momentaufnahme und keine Garantie gegen jede spätere Fehlkonfiguration.

Das ZIP-Paket ist zur gezielten Weitergabe durch den Maintainer bestimmt. Es ist nicht verschlüsselt. Die Vertraulichkeitskennzeichnung ersetzt keine Zugriffskontrolle; für die Übermittlung kann der mit der Kanzlei vereinbarte geschützte Kanal verwendet werden. Der Agent hat keine Unterlagen an einen Patentanwalt oder einen anderen Empfänger versendet.

**[E7] DPMA Anmeldung eines Patents.** Offizielle Hinweise zu technischer Offenbarung und Anmeldeunterlagen; geprüft am 20. September 2026.

<https://www.dpma.de/patente/anmeldung/index.html>